iLERNA.

MEMORIA ESCRITA DEL PROYECTO

CFGS Desarrollo de Aplicaciones Multiplataforma

MedManager - Sistema de Gestión Clínica Multiplataforma

Autor: Pau Sangenis

Tutor: Bea Serrano Lucio

Fecha de entrega: 11/12/2026

Convocatoria: 1s2627

Documentos del proyecto: Enlace a la carpeta del Drive



Índice

1. Introducción 4
1.1. Problema y contexto 4
1.2. Usuarios destinatarios 4
1.3. Alcance 4
1.4. Motivación 5
1.5. Abstract 5
1.6. Objetivos propuestos (generales y específicos) 5
2. Estado del Arte 7
2.1 Planteamiento de la búsqueda y situación actual 7
2.2 Análisis de soluciones existentes 8
2.3 Comparativa de las soluciones analizadas 11
2.4. Necesidades detectadas y justificación del proyecto 13
3. Metodología usada 14
4. Tecnologías y herramientas utilizadas en el proyecto 15
5. Planificación, Diagnóstico y Contexto Laboral 17
5.1. Contexto Laboral 17
5.2. Análisis DAFO 17
5.3. Planificación Inicial 18
5.4. Planificación Real y Desviaciones 19
6. Análisis del proyecto 20
6.1. requisitos funcionales 20
6.2. requisitos no funcionales 20
6.3. Reglas de negocio 21
6.4. Actores y casos de uso 21
6.5. diagramas de Entidad-Relación 23
6.6. diagramas de Clases 25

7. Diseño del proyecto 29
7.1. Wireframe 29
8. Despliegue y pruebas 30
9. Conclusiones 31
10. Vías futuras 32
11. Bibliografía/Webgrafía 34
12. Anexos 35
12.1. Manual de Instalación 35
12.2. Manual de Usuario 35

1. Introducción

1.1. Problema y contexto

- **Necesidades:** En el sector sanitario actual, existen clínicas que siguen dependiendo de procesos manuales y documentación física para la gestión de historiales, pruebas médicas y agendas de pacientes. Esta dependencia genera problemas graves de redundancia de datos, lentitud en la atención, dificultades para compartir información de forma inmediata entre el área de recepción y los despachos médicos, y un impacto medioambiental negativo. Además, la falta de un control de acceso digitalizado dificulta la gestión segura de los datos sensibles y el control de quién accede a ellos.
- **Finalidad:** La finalidad de MedManager es resolver estas carencias creando un ecosistema digital unificado en la red local de la clínica. El sistema busca automatizar la gestión diaria y centralizar la información en una base de datos única accesible bajo demanda desde los distintos terminales. Con ello, se persigue mejorar la calidad de la asistencia, optimizar los tiempos de trabajo del personal, reforzar la seguridad en el acceso a la información privada y establecer un modelo de trabajo sostenible sin uso de papel.

1.2. Usuarios destinatarios

Está diseñada para personal de una clínica. Al tener varias aplicaciones habrá distintos destinatarios:

- Personal médico, podrán acceder a la aplicación de escritorio.
- Personal administrativo, podrán acceder a la aplicación de escritorio.
- Personal de enfermería, podrán acceder a la aplicación de Android.

1.3. Alcance

El alcance de MedManager abarca la gestión interna del flujo médico y administrativo de la clínica. El proyecto cubrirá el ciclo completo de citas, la creación y consulta de historiales médicos y la gestión de permisos según el perfil del trabajador. El sistema se limitará a la operativa clínica, por lo que no incluirá módulos de facturación, contabilidad, pasarelas de pago ni conexión con bases de datos externas estatales (como la Seguridad Social).

1.4. Motivación

La elección de un sistema de gestión clínica para este proyecto nace directamente de una experiencia personal continuada como usuario del sistema sanitario. A lo largo de los años he tenido que acudir con frecuencia a diversas clínicas como paciente, lo que me ha permitido observar de cerca su funcionamiento organizativo y despertar una gran fascinación por cómo gestionan el flujo de trabajo entre recepción, consultas y salas de atención.

Recientemente tuve la oportunidad de acudir a un centro médico altamente tecnificado e innovador. La experiencia allí fue sumamente positiva: la coordinación interna, la rapidez en la atención y la fluidez con la que circulaba la información entre los distintos profesionales demostraban las ventajas evidentes de un entorno completamente digitalizado. Esa vivencia me impulsó a plantearme el reto de crear un sistema informático que estuviera a la altura de esas nuevas tecnologías y que ofreciera ese mismo nivel de eficiencia y agilidad.

Sin embargo, en contraste con estos centros de vanguardia, es habitual encontrar todavía muchas clínicas y policlínicas cuya infraestructura digital se encuentra obsoleta o fragmentada, dependiendo aún de procesos manuales o de fichas físicas. Esta realidad representa una gran oportunidad: diseñar un software accesible, seguro y multiplataforma que permita a estas clínicas menos modernizadas dar el salto hacia una operativa ágil y contemporánea.

Desde el punto de vista académico y profesional dentro del ciclo DAM, este proyecto se convierte en el escenario idóneo para poner en práctica y consolidar de forma integral todo lo aprendido. Desarrollar una solución para este entorno exige abordar la gestión de bases de datos relacionales, el tratamiento riguroso de datos sensibles, el control de concurrencia mediante un servidor propio con Sockets y la creación de interfaces diferenciadas (escritorio en Java Swing para tareas administrativas y aplicación Android en Kotlin para el trabajo en movilidad). Con ello, no solo se busca recrear un modelo organizativo eficiente, sino también aportar una alternativa aplicable a centros que necesitan optimizar su flujo diario hacia un modelo sin uso de papel.

1.5. Abstract

En ingles, resumen de la app, finalidad, tipo, tecnologias y resultado.

1.6. Objetivos propuestos (generales y específicos)

- Objetivo General

Diseñar, desarrollar e implementar un sistema informático multiplataforma y seguro usando tanto aplicaciones de escritorio como de dispositivo móvil para la gestión de una clínica médica creando un sistema al que he nombrado MedManager, que centralice la información administrativa y asistencial en una base de datos única y que se pueda usar por distintos perfiles de usuario mediante un servidor intermedio propio.

- Objetivo Específicos

Para alcanzar el objetivo general planteado, se establecen los siguientes objetivos específicos:

- Gestión de la información y persistencia centralizada:
  - Diseñar e implementar una base de datos relacional normalizada que garantice la integridad referencial y el almacenamiento persistente de pacientes, citas médicas, personal sanitario e historiales clínicos.
  - Centralizar el acceso a la base de datos a través de un servidor backend concurrente (multihilo) mediante Sockets TCP/IP, evitando la exposición directa de la base de datos a las aplicaciones cliente.
- Operativa administrativa y médica (Aplicación de Escritorio):
  - Proporcionar un sistema de autenticación seguro basado en roles de usuario (médico y administrativo).
  - Permitir al personal administrativo la gestión completa (altas, bajas lógicas, modificaciones y consultas) de los datos de pacientes y la programación de citas médicas evitando solapamientos horarios.
  - Facilitar al personal facultativo la consulta de su agenda de consultas y la incorporación de diagnósticos y observaciones en el historial clínico de los pacientes.
- Asistencia en movilidad (Aplicación Móvil):
  - Proporcionar un sistema de autenticación seguro basado en roles de usuario (enfermero).
  - Permitir al personal de enfermería acceder de forma rápida y autenticada a la ficha esencial del paciente desde un dispositivo móvil Android.
  - Registrar observaciones previas y notas de cribaje de enfermería en las citas del día para su consulta por parte del personal médico desde los puestos de escritorio.
- Seguridad y protección de datos:
  - Implementar mecanismos de protección de credenciales mediante algoritmos (BCrypt), control de sesiones por rol en el servidor y acceso de mínimo privilegio a la base de datos, contribuyendo a las medidas técnicas de privacidad exigidas para datos sensibles.

2. Estado del Arte

2.1 Planteamiento de la búsqueda y situación actual

Para el diseño de la aplicación de MedManager, la base de datos, la aplicación de escritorio, la aplicación móvil y la comunicación, se ha realizado una búsqueda real en el mercado de gestión de clínicas y centros médicos. Esta búsqueda tiene el objetivo de ver cómo funcionan las aplicaciones reales en el sector y poder compararlas con mis ideas para ver si son eficaces o no, pudiendo cubrir problemas que tengan estas y aportando alguna innovación o mejora si es posible. Con este análisis sabremos qué funciones tendrá nuestro proyecto.

La situación actual de las clínicas pequeñas y medianas es que han de dejar de usar el papel, ya que es más eficiente y rentable tener centralizada en un servidor toda la información para poder ser consultada rápidamente y que pueda cumplir con las normativas e inspecciones.

Para la búsqueda de software específico he usado el portal https://www.capterra.es/ (Capterra, 2026), ya que ofrece todo tipo de software disponible y explica qué nos puede aportar. Se puede apreciar que hay dos tendencias para el sector de clínicas:

- **Programas centrados en grandes hospitales:** Incluyen todo tipo de módulos parecidos a un ERP donde se gestionan facturación, contabilidad, resultados médicos, citas, pacientes, etc. Normalmente, este tipo de software, al abarcar todo, suele ser caro y difícil tanto de implementar como de actualizar y usar.
- **Programas que usan modelo SaaS (web en la nube):** Son ligeros, ya que no precisan una instalación y suelen usarse desde cualquier dispositivo, pero te obligan a contratar un mantenimiento y una suscripción periódica.

Teniendo en cuenta estas dos opciones, mi planteamiento de proyecto está enfocado a que no sea tan pesado como los pensados para grandes instalaciones ni obligue a pagar una suscripción a un modelo SaaS, sino que esté centrado en citas, informes y pacientes, evitando que la empresa tenga que contratar módulos o servicios adicionales que no necesita.

2.2 Análisis de soluciones existentes

Usando la web comentada he encontrado estas cuatro aplicaciones que me han servido para comparar sus prestaciones y funcionalidades

1. SoftClinic GenX



Esta aplicación cuenta con varios tipos de software en su página web centrados en la gestión médica; tiene versiones para clínicas, laboratorios, farmacias y hospitales, está dividido por módulos y es usado en más de 45 países. Centrándonos en la parte de clínicas, nos definen sus características:

- Citas y programación de pacientes.
- Gestión de pacientes ambulatorios (OPD).
- Facturación médica, reclamaciones y contabilidad.
- Notas de enfermería y prescripción.
- Gestor de archivos e informes.
- Control de inventario.
- Base de datos en la nube y telemedicina.

Podemos concluir con las siguientes ventajas y limitaciones:

- **Ventajas:** Es muy completo, cumple con la normativa HIPAA y cuenta con versión de escritorio además de en la nube.
- **Limitaciones:** Está pensado más como un ERP grande para hospitales, no destaca en la gestión directa de documentos según los revisores y resulta costoso si se quieren todos los módulos

2. Archivex



En esta ocasión he podido descargar y probar el producto directamente, ya que ofrecen una demo gratuita de 7 días. Es un gestor de clínicas español que se instala en el ordenador y permite:

- Configuración general de la clínica, horarios y especialidades.
- Agendar citas y programación de médicos.
- Gestión de pacientes e historia clínica electrónica.
- Facturación médica, contabilidad y proveedores.
- Campañas de marketing, automatizaciones y avisos por SMS.
- Control de stock y familias de productos



Podemos concluir con las siguientes ventajas y limitaciones:

- **Ventajas:** Al instalarse en el ordenador la interfaz va muy fluida, tiene un menú lateral muy cómodo (Calendario, Pacientes, Contabilidad) y permite organizar muy bien las especialidades y la seguridad de accesos.
- **Limitaciones:** Incluye demasiadas opciones comerciales y financieras (marketing, bonos, contabilidad, stock y proveedores) que complican el programa para el personal sanitario. Además, funciona bajo suscripción de pago tras la prueba de 7 días.

3. Clinic Cloud



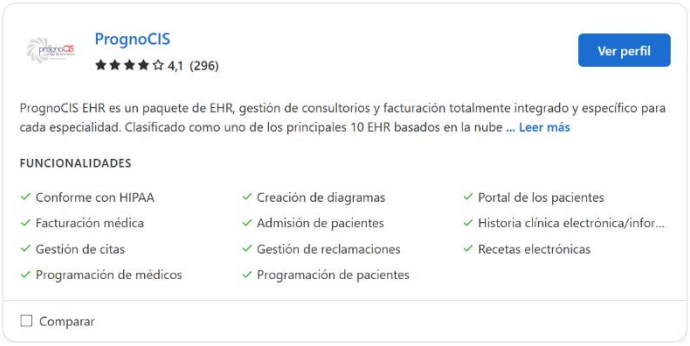
Es un software «todo en uno» pensado para la gestión integral de pacientes en una única plataforma mediante el uso de la nube, pudiendo acceder desde cualquier lugar con un navegador web. Reduce las tareas administrativas y permite la personalización. En este gestor web puedes:

- Configurar el centro médico.
- Gestionar calendario, citas y disponibilidad de agendas.
- Gestión de registros de pacientes y sus historiales clínicos digitales.
- Facturación médica, pagos, caja y stock.
- Contar con asistente virtual y diseño modular.

Podemos concluir con las siguientes ventajas y limitaciones:

- **Ventajas:** Es muy fácil de usar desde cualquier dispositivo sin instalar nada y centraliza muy bien las citas y los historiales digitales.
- **Limitaciones:** Depende al 100 % de tener conexión a Internet (si se cae la red no se puede trabajar), obliga a pagar una cuota mensual por usar su nube.

4. PrognoCIS



Es un paquete de software basado en la nube muy centrado en la historia clínica electrónica (EHR), la gestión de consultorios y la facturación integrada adaptada a distintas especialidades médicas. Según su ficha técnica, permite:

- Gestión de citas, admisión de pacientes y programación de médicos.
- Historia clínica electrónica (EHR), creación de diagramas e informes.
- Portal específico para los pacientes y recetas electrónicas.
- Facturación médica y gestión de reclamaciones de seguros.
- Cumplimiento con la normativa de seguridad sanitaria HIPAA.

Podemos concluir con las siguientes ventajas y limitaciones:

- **Ventajas:** Está muy bien valorado en la parte médica (informes, recetas y cumplimiento de seguridad HIPAA) y separa bien la programación de médicos y pacientes.
- **Limitaciones:** Al igual que otros sistemas en la nube, requiere suscripción de pago, depende de conexión externa a Internet y mete módulos pesados de facturación y reclamaciones que no entran en el alcance de nuestro proyecto.

2.3 Comparativa de las soluciones analizadas

En este apartado compararé las características que tienen las aplicaciones analizadas con las características que quiero aplicar en mi proyecto y cuáles podría adquirir habiendo analizado soluciones ya existentes.

Solución analizada	Qué hace bien	Limitaciones	Qué aprendo para mi proyecto
SoftClinic GenX	Muy completo, cumple seguridad médica y tiene versión de escritorio.	Demasiado grande (tipo ERP) y costoso.	Crear una aplicación de escritorio estable en Java Swing, pero centrada solo en la parte clínica sin contabilidad.
Archivex	Interfaz de escritorio muy clara con menú lateral y buena gestión de especialidades.	Exceso de opciones de marketing, stock y contabilidad; pago tras 7 días de prueba.	Coger la idea de su diseño limpio por secciones (Pacientes, Calendario, Especialidades) para las pantallas de escritorio.
Clinic Cloud	Gestión de citas e historiales muy intuitiva y accesible.	Funciona solo en navegador web, depende de Internet y exige suscripción mensual.	Mantener una gestión de citas e informes sencilla, pero guardando los datos en un servidor local propio (MySQL).
PrognoCIS	Muy enfocado en la historia clínica, informes y privacidad de datos.	Basado en la nube externa y mezclado con facturación y reclamaciones.	Dar prioridad a la seguridad de los datos médicos (usando contraseñas con hash y permisos por rol).

Criterio	SoftClinic GenX	Archivex	Clinic Cloud	PrognoCIS	MedManager
Despliegue	Nube / Escritorio	Escritorio local	Nube (SaaS Web)	Nube (SaaS Web)	**Red Local (Servidor TCP + MySQL)**
Clientes nativos	Escritorio / Web	Escritorio	Navegador Web	Navegador Web / App	**Escritorio (Java Swing) + App Android**
Enfoque de Enfermería	Módulo dentro de ERP	No específico	Web generalista	Orientado a consulta	**App Android dedicada a cribaje/notas**
Módulos financieros	Sí (Facturación/ERP)	Sí (Stock/Marketing)	Sí (Caja/Pagos)	Sí (Seguros/Cobros)	**No (Exclusivo operativa clínica)**
Dependencia de Internet	Según modalidad	Baja	Total (100% online)	Total (100% online)	**Ninguna (Funciona en LAN local)**
Marco de privacidad	HIPAA	RGPD / LOPDGDD	RGPD / LOPDGDD	HIPAA	**Medidas técnicas RGPD (LAN + Hash)**
Modelo de coste	Licencia / Suscripción	Suscripción tras demo	Cuota mensual	Suscripción mensual	**Sin cuotas**

2.4. Necesidades detectadas y justificación del proyecto

Tras revisar y analizar estas aplicaciones, se ve claramente que una clínica pequeña o mediana se encuentra siempre con los mismos inconvenientes:

- Pagar por funciones que no van a usar: Casi todos los programas meten por defecto módulos de contabilidad, facturación de seguros, control de stock de almacén o campañas de marketing por SMS. Esto encarece el producto y llena la pantalla de menús que ni el médico ni el enfermero necesitan.
- Dependencia de Internet y de empresas externas: Al trabajar en la nube (SaaS), la clínica está obligada a pagar una cuota todos los meses y a guardar los datos sensibles de sus pacientes en servidores ajenos. Además, si un día falla Internet en el centro, no se puede trabajar.
- Falta de una App Android dedicada a cribaje/notas: Los programas analizados están pensados para usarse sentado frente al ordenador en recepción o en la consulta médica, pero no ofrecen una aplicación móvil sencilla para que los enfermeros consulten la ficha rápida de un paciente o anoten observaciones mientras se mueven por la clínica.

Justificación de MedManager:

- Para solucionar estos puntos, mi proyecto MedManager propone una alternativa directa y sin costes de suscripción: Se centra exclusivamente en lo necesario para el trabajo diario: gestión de pacientes, programación de citas, informes médicos, dejando fuera la facturación y la contabilidad.
- Utiliza un servidor propio en Java con Sockets TCP/IP conectado a una base de datos MySQL dentro de la misma red local de la clínica, de modo que los datos están seguros en el propio centro y las aplicaciones no atacan directamente a la base de datos.
- Separa el trabajo en dos aplicaciones adaptadas a cada puesto: una aplicación de escritorio en Java Swing para administrativos y médicos, y una aplicación móvil en Android (Kotlin) para el personal de enfermería.

3. Metodología usada

(2-4 páginas)

4. Tecnologías y herramientas utilizadas en el proyecto

(2-3 páginas)

He organizado las tecnologías y herramientas según la parte del proyecto en la que se han utilizado. En cada bloque distingo las **tecnologías** (lenguajes, librerías y componentes que forman parte del producto) de las **herramientas** (programas que he empleado para desarrollarlo, diseñarlo y documentarlo).

4.1. Aplicación de escritorio

- Tecnologías:
  - Java
  - Java Swing (interfaz gráfica)
  - Sockets (cliente TCP/IP).
- Herramientas:
  - Eclipse.

4.2. Aplicación Android

- Tecnologías:
  - Kotlin
  - corrutinas (operaciones de red sin bloquear la interfaz)
  - Sockets (cliente TCP/IP).
- Herramientas:
  - Android Studio
  - Figma (diseño de pantallas).

4.3. Servidor

- Tecnologías:
  - Java
  - Sockets con un hilo por cliente
  - Hibernate (ORM)
  - BCrypt (hash de contraseñas)

    - JUnit (pruebas unitarias).
- Herramientas:
    - Eclipse.

4.4. Base de datos

- Tecnologías:
    - SQL.
- Herramientas:
    - MySQL Workbench
    - Draw.io (diagrama E-R).

4.5. Gestión y documentación

- Tecnologías y métodos:
    - Kanban para el seguimiento de tareas.
- Herramientas:
    - Miro [indica el uso],
    - Excel (planificación y diagrama de Gantt)
    - Git/GitHub, para el control de versiones

5. Planificación, Diagnóstico y Contexto Laboral

5.1. Contexto Laboral

Este proyecto está diseñado para su implantación en una clínica médica o centro de salud.

Perfiles profesionales que intervienen y el valor que aporta:

- **Usuarios administrativos:** Utilizarán la aplicación de escritorio (Java Swing) para la gestión completa (altas, modificaciones, consultas y bajas lógicas) de pacientes y la programación de la agenda de citas médicas.
- **Usuarios Medicos:** Utilizarán la aplicación de escritorio (Java Swing) para consultar su agenda diaria, revisar las observaciones previas de enfermería y redactar o consultar los informes del historial clínico
- **Usuarios enfermeros:** El personal de enfermería utilizaran la aplicación móvil (Android) para poder hacer un cribaje o asistir al paciente de forma informativa.
- **Perfiles profesionales:** El desarrollo de este ecosistema requiere perfiles de Administrador de Bases de Datos (MySQL), Desarrollador Backend (Servidor Java Multihilo y Sockets) y Desarrollador Frontend/Móvil (Java Swing y Android/Kotlin).
- **Valor aportado:** Aporta una centralización segura de los datos médicos de los pacientes. Al utilizar una arquitectura de Sockets, los usuarios no atacan la base de datos directamente, añadiendo una capa de seguridad y control de concurrencia fundamental en entornos sanitarios.

5.2. Análisis DAFO

Análisis DAFO aplicado al proyecto:

- **Fortalezas (Internas):** Uso de conexiones mediante Sockets que proporcionan un control total sobre el flujo de datos. Implementación de hilos (multithreading) tanto en el servidor como en la app móvil (Coroutines) para evitar bloqueos en la interfaz.
- **Debilidades (Internas):** La gestión manual de Sockets a bajo nivel y el mapeo ORM con Hibernate incrementan la complejidad del código y requieren un control de excepciones exhaustivo.
- **Oportunidades (Externas):** Alta demanda de digitalización y eliminación de papel en clínicas médicas.

- **Amenazas (Externas):** Existencia de un mercado saturado de software de gestión hospitalaria (HIS) estándar y la necesidad de cumplir con normativas estrictas de protección de datos (RGPD).

5.3. Planificación Inicial

Diagrama de Gantt inicial que representa la planificación estimada:

TAREA	ASIGNADO A	PROGRESO	INICIO	FIN
Documentación y Diseño			28-9-26	11-12-26
Tarea 1: Introducción, Objetivos y Estado del Arte	Pau	0%	28-9-26	2-10-26
Tarea 2: Metodología, Tecnologías y Planificación	Pau	0%	5-10-26	9-10-26
Tarea 3.1: Definición de requisitos funcionales y no funcionales	Pau	0%	12-10-26	13-10-26
Tarea 3.2: Diagramas de Casos de Uso	Pau	0%	14-10-26	15-10-26
Tarea 3.3: Creación del Diagrama Entidad-Relación (E-R)	Pau	0%	16-10-26	19-10-26
Tarea 3.4: Creación del Diagrama de Clases y Wireframes	Pau	0%	20-10-26	21-10-26
Tarea 4: Despliegue, Pruebas y Conclusiones	Pau	0%	3-12-26	4-12-26
Tarea 5: Bibliografía, Manual de instalación y Manual de usuario	Pau	0%	7-12-26	8-12-26
Tarea 6: Preparación del vídeo y entrega final	Pau	0%	9-12-26	11-12-26
Base de Datos			22-10-26	30-10-26
Creación del script DDL (Tablas, PK, FK)	Pau	0%	22-10-26	23-10-26
Restricciones de unicidad (citas) e integridad referencial	Pau	0%	26-10-26	27-10-26
Creación de usuarios y asignación de permisos mínimos	Pau	0%	28-10-26	28-10-26
Generación de datos de prueba (Scripts Inserts)	Pau	0%	29-10-26	29-10-26
Despliegue local (XAMPP/MySQL) y verificación	Pau	0%	30-10-26	30-10-26
Servidor Java, Seguridad y Pruebas Unitarias			2-11-26	13-11-26
Configuración Eclipse e Hibernate	Pau	0%	2-11-26	3-11-26
Mapeo ORM (Entidades Java a MySQL)	Pau	0%	4-11-26	6-11-26
Bucle ServerSocket multihilo	Pau	0%	9-11-26	10-11-26
Protocolo Sockets, seguridad Hash/Roles y pruebas JUnit	Pau	0%	11-11-26	13-11-26
Aplicación Escritorio (Java Swing)			16-11-26	24-11-26
Diseño de ventanas en Java Swing (JFrame/JPanel)	Pau	0%	16-11-26	18-11-26
Conexión Socket Cliente al Servidor	Pau	0%	19-11-26	20-11-26
Módulos de Pacientes, Agenda de Citas e Historial Clínico	Pau	0%	23-11-26	24-11-26
Aplicación Móvil Android y Pruebas de Integración			25-11-26	4-12-26
Diseño de layouts XML e interfaz en Kotlin	Pau	0%	25-11-26	27-11-26
Conexión Socket en hilo secundario (Coroutines)	Pau	0%	30-11-26	1-12-26
Vistas de Ficha Rápida, Observaciones y Pruebas de Integración	Pau	0%	2-12-26	4-12-26

Como se detalla en el cronograma (image_d1c204.png), el ciclo de vida del desarrollo se divide en los siguientes bloques secuenciales:

- **Documentación y Diseño (28-09-2026 al 11-12-2026):** Fase transversal que comprende el análisis de requisitos, diseño de diagramas (E-R, Relacional, Casos de Uso y Clases), elaboración de *wireframes*, redacción de manuales y grabación del vídeo de defensa final.
- **Base de Datos (22-10-2026 al 30-10-2026):** Creación del script DDL en MySQL (XAMPP), definición de restricciones de unicidad y de formato, y triggers de integridad de citas, configuración del usuario de conexión con permisos mínimos y carga de datos de prueba.
- **Servidor Java, Seguridad y Pruebas Unitarias (02-11-2026 al 13-11-2026):** Configuración de Hibernate (ORM), programación del servidor ServerSocket multihilo con validación de roles, implementación del *hashing* de contraseñas y pruebas unitarias con JUnit.
- **Aplicación Escritorio Java Swing (16-11-2026 al 24-11-2026):** Diseño de ventanas en código Java mediante contenedores Swing (JFrame/JPanel), integración del

cliente Socket y desarrollo de los módulos de pacientes, agenda de citas e historial clínico.
- **Aplicación Móvil Android y Pruebas de Integración (25-11-2026 al 04-12-2026):** Diseño de *layouts* XML en Android Studio, programación en Kotlin con corrutinas para la comunicación por Sockets, vistas de ficha rápida/observaciones y pruebas integrales del sistema antes del despliegue.

5.4. Planificación Real y Desviaciones

(Nota: La normativa exige explicar las desviaciones relevantes y mostrar la planificación real. Este apartado debes dejarlo con el título puesto, pero lo rellenarás al final del semestre comparando lo que planeaste en "image_d1c204.png" con lo que realmente tardaste).

6. Análisis del proyecto

(8-16 páginas)

6.1. requisitos funcionales

Requisitos Funcionales (RF):

- **RF1. Autenticación y Autorización por Roles en Servidor:** El sistema exigirá inicio de sesión validando las credenciales contra el servidor, el cual mantendrá el rol de la sesión activa (RECEPCIONISTA, MEDICO, ENFERMERO) y autorizará o denegará cada petición según dicho perfil.
- **RF2. Gestión Administrativa de Pacientes y Citas:** La aplicación de escritorio permitirá al perfil administrativo registrar altas, consultas, modificaciones y bajas lógicas (activo = false) de los datos identificativos y de contacto de los pacientes, así como programar, reprogramar o cancelar citas médicas.
- **RF3. Gestión Médica de Consultas e Historiales:** La aplicación de escritorio permitirá al médico consultar sus citas del día, visualizar las observaciones previas registradas por enfermería, redactar el informe clínico (diagnóstico y tratamiento), mantener actualizadas las alergias del paciente y actualizar el estado de la cita a REALIZADA.
- **RF4. Asistencia y Cribaje en Movilidad (Enfermería):** La aplicación móvil Android permitirá al personal de enfermería consultar los pacientes por DNI/Nombre completo, visualizar su ficha rápida (identidad y alergias) y su visita pendiente pudiendo añadir observaciones preliminares en la cita correspondiente.
- **RF5. Comunicación Intermedia Mediante Sockets:** Las aplicaciones cliente no se conectarán directamente a MySQL, sino que intercambiarán paquetes estructurados con el servidor backend a través de Sockets TCP/IP.

6.2. requisitos no funcionales

- RNF1. Seguridad y Criptografía: Las contraseñas de los usuarios deben almacenarse de forma unidireccional utilizando algoritmos hash, protegiendo así el acceso a datos sensibles.
- RNF2. Alta Concurrencia: El servidor backend Java debe estar estructurado mediante programación multihilo (Threads) para ser capaz de recibir y atender peticiones simultáneas de la app móvil y de la app de escritorio sin bloquearse.

- RNF3. Usabilidad (UX/UI): Las interfaces deben estar adaptadas a su entorno de uso: un entorno de escritorio con teclado/ratón (Java Swing) para tareas administrativas densas, y un entorno táctil ágil y rápido (Android) para consultas rápidas de pie en planta.
- RNF4. Persistencia e Integridad: La base de datos (MySQL) debe mantener una estricta integridad referencial de los datos. La conexión entre el servidor y la base de datos se gestionará de manera segura utilizando Hibernate (ORM).

6.3. Reglas de negocio

- RN**1. Unicidad horaria del facultativo:** Un médico no puede tener dos citas activas (con estado distinto de `CANCELADA`) en la misma fecha y franja horaria de 15 minutos.
- RN**2. No duplicidad de cita por paciente:** Un paciente no puede tener dos citas activas a la misma fecha y hora.
- RN**3. Privacidad y minimización de datos :** El personal administrativo solo tiene acceso a los datos identificativos y de contacto del paciente y a los datos organizativos de la cita (fecha, hora, médico, estado y un motivo breve indicado por el paciente al solicitarla). Tiene bloqueado el acceso a los datos clínicos: informes del historial, alergias y observaciones de enfermería. El servidor aplica este filtrado según el rol de la sesión, devolviendo a cada perfil únicamente los campos que le corresponden.
- RN**4. Delimitación de escritura clínica:** Los informes del historial médico y las alergias del paciente solo pueden ser creados o modificados por usuarios con rol MEDICO. El personal con rol ENFERMERO tiene acceso de lectura a la ficha del paciente (incluidas las alergias) y permiso de escritura únicamente sobre el campo de observaciones previas de la cita.
- RN**5. Ciclo de vida de las citas y conservación legal:** Los administrativos crean, reprograman o cancelan citas, mientras que el paso al estado realizada lo ejecuta el médico al cerrar la consulta. Los pacientes e historiales clínicos nunca sufren borrado físico, empleándose baja lógica para cumplir el deber legal de conservación (Ley 41/2002).

6.4. Actores y casos de uso

1. Actor: Administrativo / Recepcionista (Aplicación de Escritorio - Java Swing) Este perfil se encarga de la gestión inicial y del flujo organizativo de la clínica. Sus casos de uso principales son:

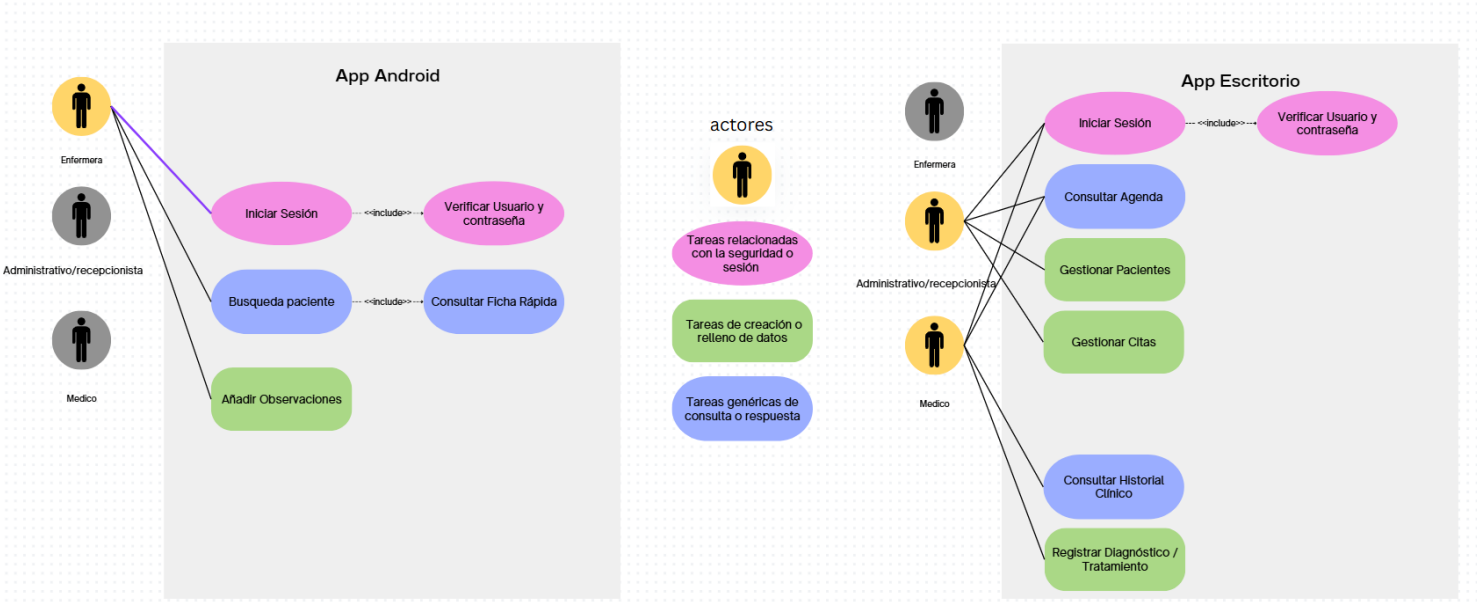
- **CU-A01. Iniciar sesión:** Acceder al sistema introduciendo sus credenciales, que serán validadas por el servidor.
- **CU-A02. Consultar Agenda:** Visualizar la agenda de citas médicas programadas
- **CU-A03. Gestión de Pacientes (CRUD):** Dar de alta a nuevos pacientes, consultar datos identificativos y de contacto, modificarlos o darlos de baja.
- **CU-A04. Gestión de Citas:** Consultar la disponibilidad de los médicos, asignar una nueva cita a un paciente, modificar la fecha/hora o cancelarla.

2. Actor: Médico (Aplicación de Escritorio - Java Swing) El personal médico requiere un entorno de escritorio para poder redactar diagnósticos extensos y revisar historiales con comodidad. Sus casos de uso principales son:

- **CU-M01. Iniciar sesión:** Acceso al sistema bajo el rol de Médico.
- **CU-M02. Consultar Agenda Propia:** Visualizar la agenda de citas médicas programadas.
- **CU-M03. Consultar Historial Clínico:** Acceder al registro médico completo de un paciente asignado.
- **CU-M04. Registrar Diagnóstico/Tratamiento:** Tras la visita, escribir el informe médico, actualizar el diagnóstico, recetar tratamientos, actualizar las alergias del paciente, guardando el informe de forma persistente en el historial y las alergias en la ficha del paciente.

3. Actor: Enfermero/a (Aplicación Móvil - Android) Este rol trabaja en movimiento (plantas, boxes, salas de extracción), por lo que interactúa exclusivamente con la aplicación móvil. Sus casos de uso principales son:

- **CU-E01. Iniciar sesión:** Acceso al sistema móvil mediante credenciales.
- ***CU-E02. Búsqueda de paciente:*** Buscar pacientes por dni o nombre***.***
- **CU-E03. Consultar Ficha Rápida:** Tras la búsqueda, visualizar la identidad, las alergias y la visita pendiente del paciente.
- **CU-E04. Añadir Observaciones:** Registrar de forma rápida cualquier nota que necesite ya sea de alguna prueba preliminar o actuación.



6.5. diagramas de Entidad-Relación

El problema es tener una base de datos para que gestione la clínica, tanto para pacientes como para trabajadores y donde se pueda consultar e introducir datos de ellos.

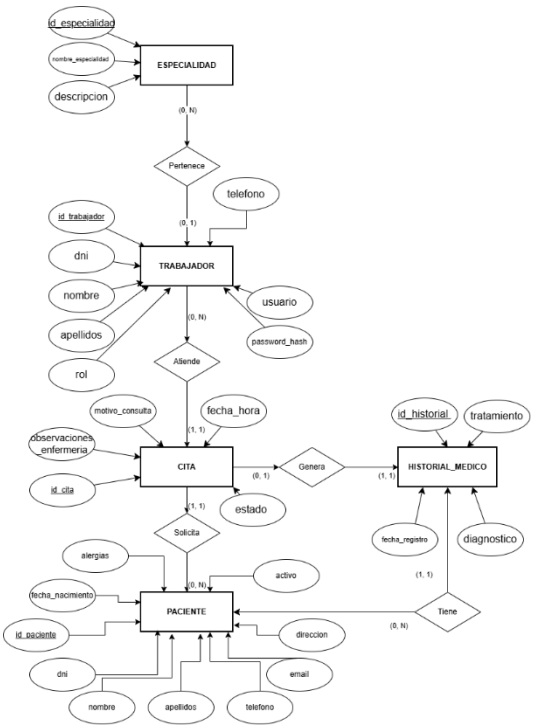
Las entidades, Los atributos y claves de cada entidad:

- **Especialidad:** id_especialidad (PK), nombre_especialidad (UNIQUE), descripcion.
- **Trabajador:** id_trabajador (PK), dni (UNIQUE), nombre, apellidos, telefono, rol (RECEPCIONISTA, ENFERMERO, MEDICO), usuario (UNIQUE), password_hash, id_especialidad (FK hacia Especialidad; obligatoria para MEDICO y nula para RECEPCIONISTA y ENFERMERO, regla garantizada mediante una restricción CHECK).
- **Paciente:** id_paciente (PK), dni (UNIQUE), nombre, apellidos, telefono, email, direccion, fecha_nacimiento, alergias, activo (BOOLEAN).
- **Cita**: id_cita (PK), id_paciente (FK hacia Paciente), id_trabajador (FK hacia Trabajador con rol MEDICO), fecha_hora, estado (PENDIENTE, REALIZADA, CANCELADA), motivo_consulta, observaciones_enfermeria.
- **Historial_Medico:** id_historial (PK), id_cita (FK UNIQUE hacia Cita), id_paciente (FK hacia Paciente), diagnostico, tratamiento, fecha_registro.

Las relaciones y Cardinalidades:

- **Paciente — Cita (0, N) / (1, 1):** Un paciente puede tener registradas desde 0 hasta N citas; una cita pertenece obligatoriamente a 1 paciente.
- **Trabajador — Cita (0, N) / (1, 1):** Un médico puede atender desde 0 hasta N citas; cada cita es atendida por 1 médico.
- **Especialidad — Trabajador (0, N) / (0, 1):** Una especialidad cuenta con 0 o varios médicos; un trabajador pertenece a 0 o 1 especialidad; solo los MEDICO la tienen.
- **Paciente — Historial_Medico (0, N) / (1, 1):** Un paciente puede tener de 0 a N registros en su historial; cada registro pertenece a 1 paciente.
- **Cita — Historial_Medico (0, 1) / (1, 1):** Una cita puede generar 0 o 1 informe médico; cada informe proviene de 1 cita.

Diagrama E-R:



Funciones y Triggers (Integridad y restricciones) de la Base de Datos:

Para garantizar la coherencia del negocio y evitar solapamientos en las agendas incluso con peticiones simultáneas, las reglas se resuelven en dos niveles: restricciones declarativas, que aplica directamente el motor de base de datos, y triggers para las comprobaciones que implican a más de una tabla.

Restricciones declarativas

1. Franjas horarias de 15 minutos (RN1). Una restricción CHECK sobre fecha_hora comprueba mediante el operador módulo que los minutos sean 00, 15, 30 o 45 y que los segundos sean 0. Si se intenta registrar una cita a las 10:07, el motor rechaza la operación.
2. Unicidad horaria del médico y del paciente (RN1 y RN2). MySQL/MariaDB no soporta índices únicos parciales de forma nativa. Para obtener el mismo efecto se añade a Cita una columna generada, fecha_hora_activa, que vale fecha_hora si el estado es distinto de CANCELADA y NULL en caso contrario. Sobre ella se definen dos restricciones UNIQUE: (id_trabajador, fecha_hora_activa) y (id_paciente, fecha_hora_activa). Como los valores NULL no colisionan entre sí, las citas canceladas liberan el hueco sin bloquear su reutilización. Al aplicarse dentro del motor, la regla se cumple tanto en altas como en reprogramaciones y es segura ante peticiones concurrentes de varios clientes.
3. Especialidad solo para médicos. Una restricción CHECK en Trabajador exige que id_especialidad sea obligatoria cuando rol = 'MEDICO' y nula en el resto de roles.

Triggers (comprobaciones entre tablas)

Al no poder expresarse con una restricción sobre una sola tabla, se definen triggers en los eventos BEFORE INSERT y BEFORE UPDATE (MySQL requiere un trigger por evento):

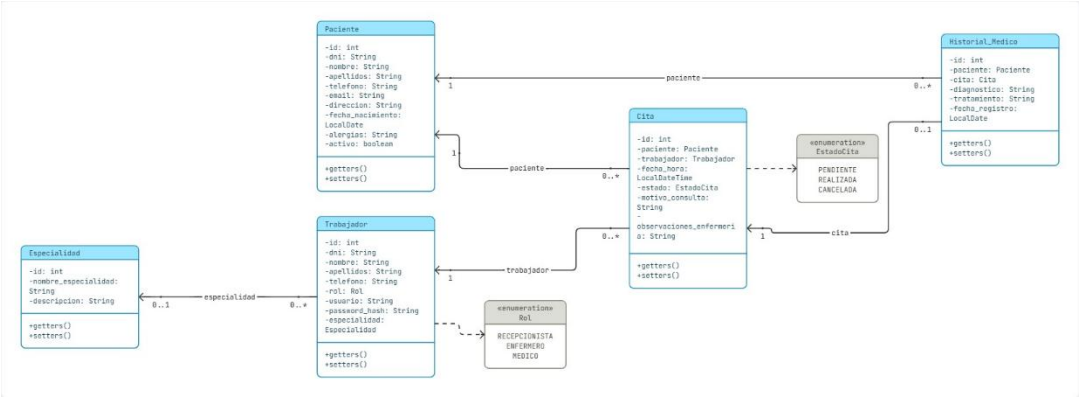
4. En Cita: comprueba que el trabajador asignado tenga rol = 'MEDICO'. Si no es así, lanza un error (SIGNAL SQLSTATE '45000') y cancela la operación.
5. En Historial_Medico: comprueba que el id_paciente del informe coincida con el paciente de la cita asociada, evitando incoherencias entre las dos rutas hacia el paciente.

En ambos casos, el servidor captura la excepción y devuelve al cliente un mensaje de error, que la aplicación muestra al usuario.

6.6. diagramas de Clases

1. Capa de Dominio (Entidades Hibernate) Representa el núcleo de la lógica de negocio y mapea exactamente las tablas de la base de datos relacional mediante el ORM Hibernate. Estas clases son compartidas en la lógica del servidor.
Las claves primarias se mapean a las columnas id_* de cada tabla, y las relaciones se representan como referencias a objetos (por ejemplo, Cita.paciente se corresponde con la columna id_paciente) mediante anotaciones de Hibernate (@JoinColumn).

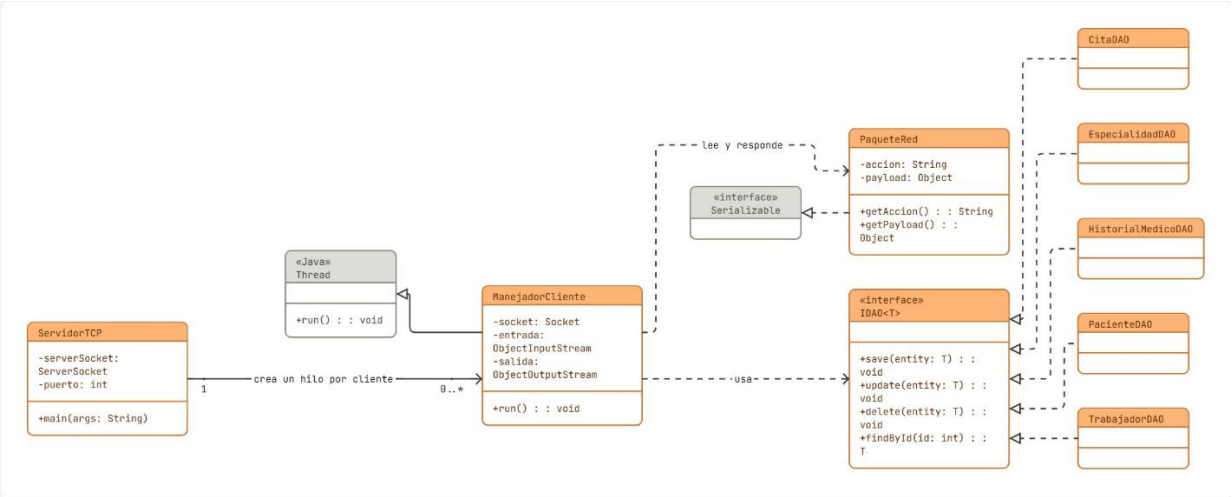
- **Clase Trabajador**: Atributos privados (id, dni, nombre, apellidos, telefono, rol, usuario, password_hash, especialidad). Métodos: Getters, Setters.
- **Clase Paciente:** Atributos privados (id, dni, nombre, apellidos, telefono, email, direccion, fecha_nacimiento, alergias, activo). Métodos: Getters y Setters.
- **Clase Cita:** Atributos privados (id, paciente, trabajador, fecha_hora, estado, motivo_consulta, observaciones_enfermeria).
- **Clase Especialidad**: Atributos privados (id, nombre_especialidad, descripcion). Métodos: Getters y Setters.
- **Clase Historial_Medico:** Atributos privados (id, paciente, cita, diagnostico, tratamiento, fecha_registro). Métodos: Getters y Setters.



2. Capa de (Backend) Se encarga de la concurrencia y la gestión de peticiones a la base de datos a través de Sockets TCP/IP. Cada hilo ManejadorCliente trabaja con su propia sesión de Hibernate, ya que las sesiones no son seguras entre hilos; la fábrica de sesiones es única y compartida por todo el servidor.

- **Clase ServidorTCP**: Contiene el método main() del servidor. Atributo ServerSocket. Se encarga de escuchar conexiones entrantes en un puerto específico de forma infinita.
- **Clase ManejadorCliente (Hereda de Thread)**: Por cada cliente que se conecta, el servidor instancia un hilo de esta clase. Atributos: Socket, ObjectInputStream, ObjectOutputStream. Método principal: run(), donde lee las peticiones del cliente y llama al DAO correspondiente.

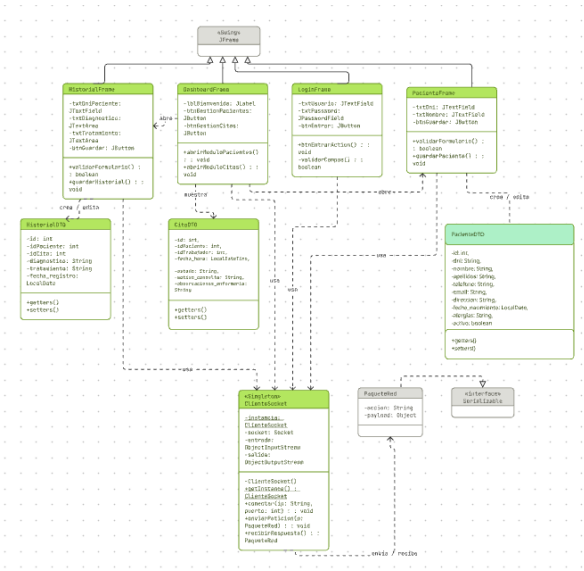
- **Clase PaqueteRed (Implementa Serializable):** Objeto que viaja por la red entre clientes y servidor. Atributos: String accion (ej: "LOGIN") y Object payload (datos).
- **Clases DAO (Data Access Object):** Implementan una interfaz genérica común (IDAO<T>). Son PacienteDAO, TrabajadorDAO, CitaDAO, EspecialidadDAO e HistorialMedicoDAO. Interactúan con Hibernate mediante las operaciones básicas de la interfaz (save(), update(), findById()) y consultas específicas según la entidad: búsqueda de pacientes por DNI o nombre, búsqueda de trabajadores por usuario (autenticación) y consulta de la agenda de un médico por fecha. En las entidades sujetas a conservación legal (Paciente e Historial_Medico) no se realiza borrado físico: la baja se implementa como una actualización del estado (activo = false), conforme a RN5.



3. Capa de Cliente (Escritorio Java Swing) Gestiona la interfaz de usuario, valida los datos de entrada y envía las peticiones al servidor.

- **Clase ClienteSocket**: Patrón Singleton. Mantiene la conexión abierta con el servidor mediante Sockets. Contiene los métodos principales para establecer la conexión, enviar objetos y recibir respuestas.
- **Clases DTO (Data Transfer Object):** Como PacienteDTO, HistorialDTO y CitaDTO. Son clases locales simplificadas que se utilizan únicamente para encapsular y estructurar los datos extraídos de los formularios visuales antes de enviarlos por la red.
- **Clase PaqueteRed:** Implementa Serializable. Es el objeto que se envía a través del ClienteSocket, el cual contiene un String con la acción a realizar (ej. "GUARDAR_PACIENTE") y un Object (payload) con el DTO correspondiente.

- **Vistas Principales y de Gestión (Frames):** Clases que heredan de contenedores nativos de Swing (JFrame o JPanel), tales como LoginFrame, DashboardFrame, PacienteFrame e HistorialFrame. Estas clases contienen la maquetación de la interfaz generada en código Java puro y los escuchadores de eventos (ActionListeners) que capturan las interacciones del usuario, instancias los DTOs y usan el ClienteSocket para enviar la información al backend.



4. Capa de Cliente Móvil (Android / Kotlin)

7. Diseño del proyecto

(6-14 páginas)

7.1. Wireframe iniciales

Wireframe App Android (Enfermera):

- Pantalla de Login.
- Pantalla principal con busqueda de paciente
- Pantalla del resultado de la busqueda (Ficha rápida del paciente).
- Observaciones de pacientes atendidos

BOCETOS FIGMA

Wireframe App Escritorio (Administrativo/Médico):

- Pantalla de Login.
- Menú principal (Dashboard) con el panel lateral de navegación.
- Pantalla de gestión de informes.

BOCETOS FIGMA

7.2. Arquitectura

7.3. Estructura interna

7.4. Interfaz

1. App android
2. App escritorio

8. Despliegue y pruebas

(4-10 páginas)

9. Conclusiones

(1-2 páginas)

10. Vías futuras

(1-2 páginas)

Como se ha justificado durante el análisis, el proyecto **MedManager** se ha centrado en desarrollar un Producto Mínimo Viable (MVP) robusto, seguro y enfocado estrictamente en la operativa clínica interna. Sin embargo, la arquitectura cliente-servidor basada en Sockets y la base de datos relacional han sido diseñadas con alta escalabilidad, lo que permite planificar las siguientes vías de mejora y evolución del software, ordenadas por prioridad de implementación:

1. Módulo de identificación por códigos QR (Prioridad Alta - Corto plazo) Como evolución técnica inmediata tras el desarrollo del MVP, se plantea la integración de un ecosistema de códigos QR. Consistiría en programar la generación e impresión de códigos QR desde la aplicación de escritorio (Java Swing) en el mostrador de recepción, y habilitar el lector de cámara en la app móvil (Android / Kotlin) para que el personal de enfermería pueda escanear la pulsera del paciente y cargar su historial médico en pantalla de forma instantánea.

2. Gestión de cuadrantes y horarios laborales (Prioridad Media) Aunque el sistema actual vincula de forma efectiva las citas a los facultativos médicos, se prevé incorporar en el futuro una tabla específica de control horario en la base de datos. Esto permitiría a la administración de la clínica generar cuadrantes de turnos (jornadas de mañana, tarde, guardias o festivos) y evitar que se puedan asignar citas en franjas donde el especialista no tiene turno activo.

Para mayor escalabilidad del aplicativo podría:

1. Portal Web para el paciente (Prioridad Comercial - Largo plazo) Actualmente, las citas deben ser introducidas manualmente por el personal administrativo al recibir llamadas o visitas. Una evolución natural del ecosistema sería desarrollar un portal web público e integrarlo al sistema central. A través de esta plataforma, los clientes podrían iniciar sesión de forma segura, consultar la disponibilidad de los médicos, solicitar o cancelar sus propias citas online y descargar sus informes o resultados médicos en formato PDF, descongestionando el volumen de trabajo del mostrador de recepción.

2. Integración de un módulo CRM (Prioridad Comercial - Largo plazo) Para dotar a la clínica de una herramienta de fidelización y seguimiento, se proyecta el desarrollo de un módulo interno CRM (*Customer Relationship Management*). Este módulo utilizaría los datos del historial y de contacto de los pacientes para automatizar procesos clave, tales como: el envío de recordatorios de citas por correo electrónico o SMS para reducir el absentismo, la

programación de avisos automatizados para revisiones médicas anuales, o la segmentación de pacientes para campañas de prevención de salud específicas.

11. Bibliografía/Webgrafía

(1-2 páginas)

12. Anexos

1. Manual de Instalación

2. Manual de Usuario

